

Gradient-Based Optimisation to Address Posterior Variance Collapse in Variational Bayesian Regression

David Ewing (82171165)

3 May 2026

Abstract

Mean-field Variational Inference (VI) provides a computationally efficient approximation to Bayesian posterior inference, but it can systematically underestimate posterior uncertainty. This failure mode is the primary concern of this report and is referred to as *posterior variance collapse*. In Bayesian regression, posterior variance collapse appears when the variational covariance matrix for the regression coefficients is too small, producing posterior intervals that are narrower than the uncertainty justified by the data and model.

The primary objective of this report is to investigate posterior variance collapse using the gradient-based optimisation methods taught in ENEL445. VI recasts Bayesian posterior computation as an optimisation problem through maximisation of the Evidence Lower Bound (ELBO), making it a natural setting for applying gradient ascent, Newton’s method, and BFGS.

To support this objective, the numerical study is designed as a complete inference-and-optimisation pipeline. Stochastic data are generated from known Bayesian regression models, and a Gibbs sampler is used as a posterior-reference method. The mean-field variational approximation is then formulated and the ELBO is derived as the objective function. This ELBO objective is then solved using the Coordinate Ascent Variational Inference (CAVI) and the gradient-based methods identified above.

The main objective is to determine whether successful ELBO optimisation also produces reliable posterior uncertainty. This distinction is important because posterior variance collapse can remain even when the ELBO has converged. The methods are therefore compared using both optimisation diagnostics, such as convergence behaviour, final ELBO value, step-size behaviour, and computational cost, and posterior-uncertainty diagnostics, especially posterior standard-deviation ratios relative to the Gibbs reference.

For the gradient-based methods, the report states the optimisation variables, constraints, entry conditions, search directions, step-size rules, and exit conditions. The variational parameters include $\boldsymbol{\mu}_\beta$, $\boldsymbol{\Sigma}_\beta$, a_e , and b_e , with Cholesky and log-space reparameterisations used to keep the covariance matrix positive definite and the Gamma parameters positive throughout optimisation. For line-search methods, the search direction must satisfy the ascent condition $\nabla \text{ELBO}(\boldsymbol{\phi}^{(t)})^T \mathbf{d}^{(t)} > 0$. Gradient ascent uses Armijo backtracking, while BFGS uses Wolfe conditions so that the curvature condition $\mathbf{y}_t^T \mathbf{s}_t > 0$ is maintained for the inverse-Hessian approximation.

Two numerical test cases are used: a linear Bayesian regression problem and a quadratic Bayesian regression problem. The quadratic case is treated through feature expansion, so the model remains linear in the coefficient vector while including the nonlinear feature x^2 . Code for generating both test cases and applying the optimisation methods is included in the report, with the full implementation structure provided in the appendix.

The primary objective of this report is to investigate posterior variance collapse using the gradient-based optimisation methods taught in ENEL445. VI recasts Bayesian posterior

computation as an optimisation problem through maximisation of the Evidence Lower Bound (ELBO), making it a natural setting for applying gradient ascent, Newton’s method, and BFGS. Coordinate Ascent Variational Inference (CAVI) is used as the analytical baseline because it exploits the conjugate structure of the Bayesian regression model.

The ELBO is derived in closed form for Bayesian regression under a mean-field Normal–Gamma approximation. For the gradient-based methods, the report states the optimisation variables, constraints, entry conditions, search directions, step-size rules, and exit conditions. The variational parameters include $\boldsymbol{\mu}_\beta$, $\boldsymbol{\Sigma}_\beta$, a_e , and b_e , with Cholesky and log-space reparameterisations used to keep the covariance matrix positive definite and the Gamma parameters positive throughout optimisation.

For line-search methods, the search direction $\mathbf{d}^{(t)}$ must satisfy the ascent condition

$$\nabla \text{ELBO}(\boldsymbol{\phi}^{(t)})^T \mathbf{d}^{(t)} > 0,$$

ensuring that the line search is well-posed for maximising the ELBO. Gradient ascent uses Armijo backtracking, while BFGS uses Wolfe conditions so that the curvature condition $\mathbf{y}_t^T \mathbf{s}_t > 0$ is maintained for the inverse-Hessian approximation. Exit conditions are applied uniformly using gradient norm, relative ELBO change, parameter change, and maximum-iteration criteria, with CAVI additionally monitoring the maximum relative change in its variational parameters.

Two numerical test cases are used: a linear Bayesian regression problem and a quadratic Bayesian regression problem. The quadratic case is treated through feature expansion, so the model remains linear in the coefficient vector while including the nonlinear feature x^2 . Code for generating both test cases and applying the optimisation methods is included in the report, with the full implementation structure provided in the appendix. The methods are compared by convergence behaviour, computational cost, posterior mean accuracy, and posterior variance behaviour.

Introduction

Bayesian inference provides principled uncertainty quantification but requires computing intractable posterior distributions. For most real-world models, the posterior integral,

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})}{\int p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}},$$

cannot be evaluated analytically. Variational Inference (VI) converts this inference problem into an optimisation problem, making it directly amenable to the gradient-based methods in this course.

Instead of computing the true posterior, VI finds the “best” approximation $q(\boldsymbol{\theta})$ from a tractable family $Q(\boldsymbol{\theta})$ by maximising the Evidence Lower Bound (ELBO). This transforms intractable integration into tractable optimisation—a natural testbed for the gradient-based methods.

A well-known failure mode of mean-field VI is *Posterior Variance Collapse*: the optimised approximation $q(\boldsymbol{\theta})$ is systematically overconfident, underestimating posterior uncertainty. This is a direct consequence of the KL direction minimised during ELBO maximisation, compounded by the mean-field factorisation assumption.¹

This paper addresses variance collapse via two complementary strategies,

- The first applies new optimisation methods to the *standard* ELBO objective: gradient ascent, Newton’s method, and BFGS are compared to determine whether the choice of optimiser affects dispersion quality.
- The second modifies the objective function itself, augmenting the ELBO with an entropy regularisation term $\lambda H[q(\boldsymbol{\beta})]$ to create a new objective $\mathcal{L}_{\text{reg}}(\boldsymbol{\phi}) = \mathcal{L}(\boldsymbol{\phi}) + \lambda H[q(\boldsymbol{\beta})]$ that explicitly incentivises broader posteriors.

Both the optimiser-comparison and modified-ELBO strategies are explored.

Throughout all gradient-based methods, the variational covariance $\boldsymbol{\Sigma}_{\beta}$ must remain symmetric positive definite at every iteration.² Standard gradient steps in Euclidean space can violate this requirement, leading to numerical failure. The solution is to parameterise the optimisation over the Cholesky factor $\mathbf{L}$ rather than $\boldsymbol{\Sigma}_{\beta}$ directly. Cholesky reparameterisation is a numerical implementation technique. Cholesky is the domain-specific choice for satisfying the constraint $\boldsymbol{\Sigma}_{\beta} \succ 0$ when applying those methods to this problem. The set of symmetric positive definite matrices is not a flat subspace of $\mathbb{R}^{p \times p}$: it has a curved boundary, and a straight-line gradient step can cross outside it, producing an invalid covariance matrix. Cholesky reparameterisation handles this by converting the constraint into a smooth bijection between the valid set and unconstrained $\mathbb{R}^{p(p+1)/2}$, so standard gradient steps always remain valid. It is standard practice in variational inference and Bayesian computation.

The classical foundation for the variational framework used here is least-squares regression, as developed in lectures by Prof. Roy S. Smith (Erskine Visiting Fellow, ETH Zürich, ENEL445 2026). The design matrix $\mathbf{X}$ ³ and coefficient vector $\boldsymbol{\beta}$ ⁴ follow the standard over-determined

¹See [Appendix A](#) for the full mechanistic explanation.

²This structural constraint is satisfied automatically via Cholesky reparameterisation; see [Appendix B](#).

³Denoted Φ and called the *regressor matrix* in lectures (Prof. R. S. Smith, ENEL445 2026).

⁴Denoted $\boldsymbol{\theta}$ in lectures.

formulation ($n > p^5$), so that the normal equations $(\mathbf{X}^T \mathbf{X})\boldsymbol{\beta} = \mathbf{X}^T \mathbf{y}$ have a unique solution and the Bayesian posterior mean reduces to this solution in the limit of a flat prior. The noise model adopted here—errors on $\mathbf{y}$ only, with noise precision τ_e —corresponds to the equation-error form; this is an explicit modelling assumption rather than a derived result.

0.1 Numerical Study and Required Optimisation Components

This report addresses the optimisation-method requirements by formulating variational inference for Bayesian regression as a constrained optimisation problem and applying four optimisation approaches: Coordinate Ascent Variational Inference (CAVI), gradient ascent, Newton’s method, and BFGS. Each method is described in terms of its objective function, entry conditions, search direction, step-size rule, and exit conditions. This ensures that the numerical comparison is based not only on final accuracy, but also on the optimisation behaviour of each method.

For all gradient-based line-search methods, the proposed search direction must be an ascent direction. That is, before a step is accepted, the direction $\mathbf{d}^{(t)}$ must satisfy

$$\nabla \text{ELBO}(\boldsymbol{\phi}^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (1)$$

This condition ensures that the line search is well-posed for maximising the ELBO. Gradient ascent uses the gradient itself as the search direction, while Newton’s method and BFGS use curvature information to construct more efficient directions.

The step size $\alpha^{(t)}$ is determined using line-search conditions. Gradient ascent uses Armijo backtracking to enforce sufficient increase in the ELBO. BFGS uses Wolfe conditions, combining sufficient increase with a curvature condition. The curvature condition is important because it helps ensure

$$\mathbf{y}_t^T \mathbf{s}_t > 0, \quad (2)$$

where $\mathbf{s}_t$ is the parameter-change vector and $\mathbf{y}_t$ is the gradient-difference vector. This positivity condition is required for the BFGS update to preserve positive definiteness of the approximate inverse Hessian.

Exit conditions are applied uniformly across the optimisation methods. Iteration terminates when any of the following conditions is satisfied:

- (i) Gradient norm: $\|\nabla \text{ELBO}\| < \varepsilon_g$.
- (ii) Relative ELBO change: $|\Delta \text{ELBO}|/|\text{ELBO}| < \varepsilon_r$.
- (iii) Parameter change: $\|\Delta \boldsymbol{\phi}\| < \varepsilon_\phi$.
- (iv) Maximum iterations: $t \geq t_{\max}$.

CAVI additionally monitors the maximum relative change in its variational parameters:

$$\max_i \frac{|\phi_i^{(t+1)} - \phi_i^{(t)}|}{|\phi_i^{(t+1)}|} < \varepsilon. \quad (3)$$

The numerical study is carried out using two test cases: a linear Bayesian regression problem and a quadratic Bayesian regression problem. These are introduced here only to identify the experimental scope; their full mathematical definitions and code implementations are given in Section 6. The linear case provides the baseline regression setting, while the quadratic case tests the same optimisation methods after feature expansion through the nonlinear term x^2 .

For both test cases, the report compares convergence speed, final ELBO value, computational cost, posterior mean accuracy, posterior variance behaviour, and stability of the covariance estimate. Code for generating the test cases and applying the optimisation methods is included in the report, with the full implementation provided in the appendix.

⁵Denoted d in lectures.

0.2 Bayesian Linear Regression and Variational Inference

The statistical setting used throughout the report is Bayesian regression. We examine two numerical examples: a linear regression test case and a quadratic regression test case. Both use the same Bayesian modelling and variational-inference framework; the quadratic example is handled by expanding the design matrix to include the nonlinear feature x^2 , so that the model remains linear in the coefficient vector.

0.3 Bayesian Regression Model

The Bayesian regression model is

$$\mathbf{y} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \tau_e^{-1}\mathbf{I}_n) \quad (\text{likelihood}), \quad (4)$$

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \lambda_\beta^{-1}\mathbf{I}_p) \quad (\text{prior on coefficients}), \quad (5)$$

$$\tau_e \sim \text{Gamma}(\alpha_e, \gamma_e) \quad (\text{prior on precision}). \quad (6)$$

Here, $\mathbf{X}$ is the design matrix, $\mathbf{y}$ is the response vector, $\boldsymbol{\beta}$ is the coefficient vector, and τ_e is the observation-noise precision.

0.4 Mean-Field Variational Approximation

The mean-field approximation is

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e), \quad (7)$$

$$q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta), \quad (8)$$

$$q(\tau_e) = \text{Gamma}(a_e, b_e). \quad (9)$$

The variational parameters are

$$\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e), \quad (10)$$

with dimension $d = p + p(p+1)/2 + 2$. For $p = 3$, this gives $d = 3 + 6 + 2 = 11$ parameters to optimise.

0.5 Key Notation

Symbol	Meaning
Bayesian model	
$\mathbf{X}$	Design matrix ($n \times p$)
$\mathbf{y}$	Response vector
$\boldsymbol{\beta}$	Regression coefficient vector
τ_e	Observation-noise precision
α_e, γ_e	Shape and rate of the prior on τ_e
p	Number of predictors
n	Number of observations
Variational approximation	
$q(\cdot)$	Variational approximating distribution
$\boldsymbol{\phi}$	Full variational parameter vector ($\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e$)
$\boldsymbol{\mu}_\beta$	Mean of variational posterior $q(\boldsymbol{\beta})$
$\boldsymbol{\Sigma}_\beta$	Covariance of variational posterior $q(\boldsymbol{\beta})$
a_e, b_e	Shape and rate of variational $q(\tau_e) = \text{Gamma}(a_e, b_e)$
$\mathcal{L}(\boldsymbol{\phi})$	Evidence Lower Bound (ELBO)
$H[q(\boldsymbol{\beta})]$	Differential entropy of $q(\boldsymbol{\beta})$
λ	Entropy regularisation weight
Reparameterisation	
$\mathbf{L}$	Lower-triangular Cholesky factor, $\boldsymbol{\Sigma}_\beta = \mathbf{L}\mathbf{L}^T$
$\tilde{\ell}_{ii}$	Log-diagonal entry, $L_{ii} = e^{\tilde{\ell}_{ii}}$ (ensures $L_{ii} > 0$)
$\boldsymbol{\xi}$	Unconstrained optimisation vector ($\boldsymbol{\mu}_\beta, \tilde{\ell}_{11}, \ell_{21}, \dots, \eta_a, \eta_b$)
η_a, η_b	Log-space parameters, $a_e = e^{\eta_a}, b_e = e^{\eta_b}$
Optimisation	
$\mathbf{d}^{(t)}$	Search direction at iteration t
α_t	Step size (line-search parameter) at iteration t
$\nabla \mathcal{L}$	Gradient of the ELBO w.r.t. $\boldsymbol{\xi}$
$\nabla^2 \mathcal{L}$	Hessian of the ELBO w.r.t. $\boldsymbol{\xi}$
$\mathbf{B}_t$	BFGS inverse-Hessian approximation at iteration t
Special functions	
$\psi(x)$	Digamma function, $\psi(x) = \frac{d}{dx} \log \Gamma(x)$
$\psi^{(1)}(x)$	Trigamma function, $\psi^{(1)}(x) = \frac{d}{dx} \psi(x)$
Lecture correspondence (Prof. R. S. Smith, ENEL445 2026)	
Φ	Regressor matrix = $\mathbf{X}$ in this report
θ	Coefficient vector = $\boldsymbol{\beta}$ in this report
d	Number of parameters = p in this report
$J(\theta)$	Cost function = $\mathcal{L}(\boldsymbol{\phi})$ (ELBO) in this report

1 Optimisation Problem Formulation

Having established that VI converts posterior inference into optimisation, the problem must be stated precisely before any method can be applied. This section identifies the decision variables, the objective function, and the constraints, following the standard form used throughout the course. A well-posed formulation is necessary both for implementation and for meaningful comparison across methods: gradient ascent, Newton's method, and BFGS all operate on the same objective and respect the same constraints, so differences in performance reflect the methods themselves rather than problem differences.

Decision Variables

For Bayesian linear regression with p predictors, the variational parameters (decision variables) are:

- $\boldsymbol{\mu}_\beta \in \mathbb{R}^p$: posterior mean of regression coefficients
- $\boldsymbol{\Sigma}_\beta \in \mathbb{R}^{p \times p}$: posterior covariance (symmetric positive definite)⁶
- $a_e, b_e > 0$: shape and rate parameters of the variational Gamma factor for precision

Here, a_e and b_e parameterise $q(\tau_e) = \text{Gamma}(a_e, b_e)$; the model prior on τ_e is specified in the Test Case section.

Total dimension: $d = p + \frac{p(p+1)}{2} + 2 = 11$ variables for $p = 3$ predictors.

Objective Function

Maximise the Evidence Lower Bound (ELBO):

$$\text{ELBO}(\phi) = \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] + H(q) \quad (11)$$

$$= \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] - \mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)]. \quad (12)$$

where $\phi = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$ are the variational parameters.

Constraints

- $\boldsymbol{\Sigma}_\beta \succ 0$: Covariance matrix must be positive definite (see also: positive semidefinite, $\mathbf{A} \succeq 0$)⁷
- $a_e, b_e > 0$: Gamma parameters must be positive
- Optional: Minimum variance constraints $\sigma_{\beta_j}^2 \geq \sigma_{\min}^2$ to prevent underdispersion

Constraint Handling via Reparameterisation:

To enforce $\boldsymbol{\Sigma}_\beta \succ 0$, use Cholesky decomposition:

$$\boldsymbol{\Sigma}_\beta = \mathbf{L}\mathbf{L}^T \quad (13)$$

where $\mathbf{L} \in \mathbb{R}^{p \times p}$ is lower triangular with positive diagonal entries. The unconstrained parameters are:

- $L_{ii} = e^{\ell_{ii}}$ for $i = 1, \dots, p$ (exponentiate to ensure positivity)
- $L_{ij} = \ell_{ij}$ for $i > j$ (off-diagonal entries unconstrained)

⁶A matrix $\mathbf{A}$ is **positive definite** if $\mathbf{v}^T \mathbf{A} \mathbf{v} > 0$ for every non-zero vector $\mathbf{v}$. This means that the matrix amplifies every direction — there is no direction in which it squashes things to zero or flips them negative. For a covariance matrix this means every direction in parameter space has genuine positive uncertainty. Written $\mathbf{A} \succ 0$.

⁷**Positive semidefinite** ($\mathbf{A} \succeq 0$) allows $\mathbf{v}^T \mathbf{A} \mathbf{v} = 0$ for some non-zero $\mathbf{v}$, meaning some combination of coefficients has *exactly* zero uncertainty — the distribution is flat in that direction. A valid covariance matrix only needs to be positive semidefinite, but a semidefinite (not strictly definite) matrix is non-invertible (a matrix is **invertible** if $\mathbf{A}^{-1}$ exists, meaning the equation $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a unique solution for any $\mathbf{b}$; equivalently, none of the eigenvalues are zero). The precision matrix $\boldsymbol{\Sigma}_\beta^{-1}$ appears in the ELBO and CAVI updates, so a non-invertible covariance causes those computations to fail. Such a semidefinite matrix also leads to *degenerate directions*: a **degenerate direction** is one in which the distribution has zero spread (the probability ellipsoid is flat along that direction), which corresponds to a zero eigenvalue and makes the covariance singular.

Total Cholesky parameters: $\frac{p(p+1)}{2}$ elements of $\boldsymbol{\ell}$.

To enforce $a_e, b_e > 0$, use log-space parameterisation:

$$a_e = e^{\eta_a}, \quad b_e = e^{\eta_b} \quad (14)$$

where $\eta_a, \eta_b \in \mathbb{R}$ are unconstrained.

Unconstrained parameter vector: $\boldsymbol{\xi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\ell}, \eta_a, \eta_b) \in \mathbb{R}^d$ with $d = p + \frac{p(p+1)}{2} + 2 = 11$ for $p = 3$.

Computational Characteristics

- Gradient computation: $O(np^2)$ per evaluation (dominated by matrix operations)
- Hessian computation: $O(np^2 + d^3)$ for Newton's method
- Parameters exhibit coupling (changing $\boldsymbol{\mu}_\beta$ affects optimal $\boldsymbol{\Sigma}_\beta$)
- Must handle special functions: digamma $\psi(\cdot)$, trigamma $\psi'(\cdot)$ for Gamma distribution

2 Posterior Variance Collapse

Posterior variance collapse is the main problem investigated in this report. It refers to the tendency of mean-field VI to produce posterior approximations with variances that are too small. In Bayesian regression, this appears as a covariance matrix $\boldsymbol{\Sigma}_\beta$ whose diagonal entries underestimate the uncertainty in the regression coefficients. The resulting posterior intervals are too narrow, making the model appear more confident than it should be.

This matters because Bayesian inference is not only about estimating a mean or best-fitting parameter value. It is also about quantifying uncertainty. An optimisation method may successfully maximise the ELBO while still producing a posterior approximation that understates uncertainty. This distinction is central to the numerical comparisons in this report.

2.1 Why Collapse Occurs

Mean-field VI restricts the approximating family. For example, the approximation

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta})q(\tau_e) \quad (15)$$

removes posterior dependence between the coefficient vector and the noise precision. If the true posterior contains dependence between these quantities, the mean-field approximation cannot represent it fully. Some of the uncertainty that would otherwise appear through posterior dependence is therefore lost.

The second cause is the direction of the KL divergence implied by the ELBO. Standard VI minimises

$$\text{KL}[q||p] = \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} | \mathbf{y})} d\boldsymbol{\theta}. \quad (16)$$

This objective strongly penalises the approximation when q assigns probability to regions that are poorly supported by the true posterior. It does not penalise the approximation in the same direct way for failing to cover all regions of meaningful posterior probability. The practical result is that q can become too concentrated, especially when the variational family is restricted.

2.2 Observation in This Report

The numerical experiments compare the variational posterior against a Gibbs sampler reference. The Gibbs sampler is not used as an ELBO optimiser; it is used as a posterior-uncertainty benchmark. For each coefficient, the standard-deviation ratio

$$\frac{\sigma_{\text{VB}}}{\sigma_{\text{Gibbs}}} \quad (17)$$

is used as a direct diagnostic. Ratios below one indicate that the variational approximation is narrower than the Gibbs reference.

2.3 Approaches Considered

Three approaches are considered for reducing or diagnosing posterior variance collapse.

2.3.1 Informative Prior on τ_e

A stronger prior on τ_e can limit how large the expected precision becomes. Since larger precision corresponds to smaller observation-noise variance, controlling τ_e can indirectly prevent Σ_β from becoming too small. The limitation is that this changes the statistical model and requires a defensible prior choice.

2.3.2 Minimum Variance Constraint

A direct constraint can be imposed on the diagonal entries of Σ_β :

$$\Sigma_{\beta,jj} \geq \sigma_{\min}^2, \quad j = 1, \dots, p. \quad (18)$$

This prevents individual coefficient variances from falling below a chosen threshold. The limitation is that the threshold is external to the original ELBO objective.

2.3.3 Entropy Regularisation

The most direct objective-based approach considered here is entropy regularisation:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda H[q(\beta)]. \quad (19)$$

For $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, the entropy is

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \Sigma_\beta) \quad (20)$$

$$= \frac{p}{2} (1 + \log 2\pi) + \frac{1}{2} \log \det(\Sigma_\beta). \quad (21)$$

The regularised gradient with respect to the covariance matrix gains the additional term

$$\nabla_{\Sigma_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\Sigma_\beta} \mathcal{L} + \frac{\lambda}{2} \Sigma_\beta^{-1}. \quad (22)$$

This term rewards broader covariance estimates and directly counters the tendency toward overly narrow posterior approximations.

3 Constraint Handling via Reparameterisation

The variational parameters are not free. Σ_β must remain symmetric positive definite throughout optimisation,⁸ and a_e, b_e must remain strictly positive.⁹ Four strategies exist for handling these constraints in a gradient-based optimiser: ignore them (the algorithm crashes when $\log|\Sigma_\beta|$ becomes undefined); add a penalty term (introduces a tuning parameter and ill-conditioning as $\rho \rightarrow \infty$); project back to the feasible set after each step (costs $O(p^3)$ per iteration and is non-differentiable at the boundary, breaking BFGS); or reparameterise so the constraints are encoded structurally and cannot be violated. This work uses reparameterisation throughout—it requires no tuning parameters, no outer loops, and produces a smooth unconstrained problem fully compatible with gradient ascent, Newton, and BFGS. Two transformations are applied: one for the matrix constraint on Σ_β , and one for the scalar positivity constraints on a_e and b_e .

Cholesky Reparameterisation

Σ_β is the covariance matrix of the variational distribution $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, and must therefore remain symmetric positive definite (SPD) at every iteration. A gradient step has no awareness of this constraint: nothing stops an update from producing a matrix with a negative eigenvalue, after which $\log|\Sigma_\beta|$ and Σ_β^{-1} are undefined and the algorithm crashes.

The transformation is $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular. Optimising over $\mathbf{L}$ instead of Σ_β directly ensures any $\mathbf{L}$ produces a valid symmetric positive semidefinite matrix by construction. However, a residual constraint remains: the diagonal entries ℓ_{ii} must be strictly positive for Σ_β to be positive definite rather than merely positive semidefinite. Three options exist: ignore the diagonal constraint (positivity can still be violated, recovering the original problem); apply a softplus function $\ell_{ii} = \log(1 + e^{u_i})$ (valid but less standard); or apply a log-space transformation $\ell_{ii} = e^{\tilde{\ell}_{ii}}$, keeping $\tilde{\ell}_{ii} \in \mathbb{R}$ unconstrained. The log-space option is used here. The parameter count is unchanged at $p(p+1)/2$ free entries: p log-diagonal entries $\tilde{\ell}_{ii}$ and $p(p-1)/2$ unconstrained off-diagonal entries $\ell_{ij}, i > j$.

Log-Space Reparameterisation

The Gamma shape and rate parameters $a_e, b_e > 0$ carry a scalar positivity constraint. The same reparameterisation principle applies: define $\eta_a = \log a_e$ and $\eta_b = \log b_e$, so that $a_e = e^{\eta_a}$ and $b_e = e^{\eta_b}$. Since the exponential function maps $\mathbb{R}$ onto $(0, \infty)$, the optimiser moves η_a, η_b freely over the real line and the positivity constraint is structurally impossible to violate. This is the scalar analogue of the Cholesky diagonal transformation, and the same principle as substituting $x = e^t$ to eliminate $x > 0$ in single-variable optimisation.

After both transformations, the effective optimisation variables are $(\mu_\beta, \tilde{\ell}_{11}, \ell_{21}, \tilde{\ell}_{22}, \dots, \eta_a, \eta_b)$ —all unconstrained reals. The quantities Σ_β, a_e, b_e are recovered on demand when evaluating the ELBO and its gradients.

4 ELBO Evaluation

Before running any optimiser, the ELBO and its analytical gradient are verified numerically. This ensures that the objective function being maximised is correctly implemented and that all reparameterisations are consistent.

⁸ Σ_β is the covariance matrix of $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$. A valid covariance matrix must be symmetric positive definite: $\log|\Sigma_\beta|$ and Σ_β^{-1} both appear in the ELBO and its gradient, so a non-positive-definite Σ_β causes immediate numerical failure.

⁹ a_e and b_e are the shape and rate of $q(\tau_e) = \text{Gamma}(a_e, b_e)$. The Gamma distribution is only defined for positive parameters; negative or zero values make $\log \Gamma(a_e)$ and $\psi(a_e)$ (digamma) undefined.

4.1 Gradient Verification

The analytical gradient $\nabla_{\theta}\mathcal{L}$ is computed with respect to all unconstrained parameters $\theta = (\mu_{\beta}, \tilde{\ell}_{11}, \ell_{21}, \tilde{\ell}_{22}, \eta_a, \eta_b)$. Each component is checked against a central finite-difference approximation:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + h e_i) - \mathcal{L}(\theta - h e_i)}{2h}, \quad h = 10^{-5}. \quad (23)$$

The relative error between the analytical and numerical gradient must satisfy $\|\nabla_{\text{analytical}} - \nabla_{\text{FD}}\|_{\infty} / \|\nabla_{\text{analytical}}\|_{\infty} \leq 10^{-5}$ for all components before any optimiser is run. The result is displayed as a bar chart (`linear_elbo_gradient_check.png`) comparing analytical and finite-difference values side by side for each unconstrained parameter. The gradient check table is saved to `results/tables/linear_gradient_check.tex`.

4.2 ELBO Landscape

A two-dimensional slice of the ELBO surface is computed by varying μ_{β_0} and μ_{β_1} over a grid while holding all other parameters fixed at their CAVI optimum. This visualises the objective surface that the gradient-based optimisers are navigating and confirms that the CAVI solution lies at the maximum. The result is saved as `linear_elbo_landscape.png`.

5 Optimisation Methods

The optimisation methods are used to investigate posterior variance collapse, not merely to solve the ELBO maximisation problem. Each method is assessed both as a numerical optimiser and as a mechanism for producing a variational posterior whose uncertainty can be compared against a Gibbs sampler reference.

5.1 CAVI Baseline

Coordinate Ascent Variational Inference (CAVI) is the analytical baseline for this conjugate Bayesian regression model. It updates each variational parameter block sequentially using closed-form solutions:

$$\Sigma_{\beta} \leftarrow \left(\frac{a_e}{b_e} \mathbf{X}^T \mathbf{X} + \lambda_{\beta} \mathbf{I}_p \right)^{-1}, \quad (24)$$

$$\mu_{\beta} \leftarrow \frac{a_e}{b_e} \Sigma_{\beta} \mathbf{X}^T \mathbf{y}, \quad (25)$$

$$a_e \leftarrow \alpha_e + \frac{n}{2}, \quad (26)$$

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X} \mu_{\beta}\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \Sigma_{\beta})]. \quad (27)$$

CAVI has monotonic ELBO increase¹⁰, linear convergence, and $O(np^2)$ cost per iteration. It provides the benchmark against which the gradient-based methods are checked.

5.2 Gradient Ascent

Gradient ascent applies the first-order update

$$\phi^{(t+1)} = \phi^{(t)} + \alpha_t \nabla \text{ELBO}(\phi^{(t)}). \quad (28)$$

¹⁰Monotonic ELBO increase means the objective is non-decreasing across iterations: $\mathcal{L}^{(t+1)} \geq \mathcal{L}^{(t)}$. In CAVI, each coordinate update maximises the ELBO with respect to one variational factor while holding the others fixed, so each step cannot decrease the ELBO.

In the unconstrained parameterisation, the update is applied to ξ rather than directly to ϕ . The search direction is the gradient, so the ascent condition is satisfied whenever the gradient is non-zero:

$$\nabla \text{ELBO}(\xi^{(t)})^T \nabla \text{ELBO}(\xi^{(t)}) > 0. \quad (29)$$

The step size is selected using Armijo backtracking so that each accepted step produces sufficient increase in the ELBO.

5.3 Newton’s Method

Newton’s method uses second-order curvature information. For maximisation, the ideal Newton direction is

$$\mathbf{d}^{(t)} = -[\nabla^2 \text{ELBO}(\xi^{(t)})]^{-1} \nabla \text{ELBO}(\xi^{(t)}). \quad (30)$$

Near a well-conditioned local maximum, Newton’s method can converge quadratically. Away from the optimum, however, the Hessian may fail to be negative definite, so the raw Newton direction may not be an ascent direction. Regularisation is therefore applied when necessary so that the search direction satisfies

$$\nabla \text{ELBO}(\xi^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (31)$$

5.4 BFGS

BFGS is a quasi-Newton method that builds an approximation to inverse Hessian information using gradient differences. Define

$$\mathbf{s}_t = \xi^{(t+1)} - \xi^{(t)}, \quad (32)$$

$$\mathbf{y}_t = \nabla \text{ELBO}(\xi^{(t+1)}) - \nabla \text{ELBO}(\xi^{(t)}). \quad (33)$$

The BFGS update requires the curvature condition

$$\mathbf{y}_t^T \mathbf{s}_t > 0 \quad (34)$$

to maintain positive definiteness of the approximate inverse Hessian. Wolfe line-search conditions are used for this reason. BFGS has $O(d^2)$ per-iteration cost and is expected to provide a practical balance between gradient ascent and Newton’s method.

5.5 Shared Entry and Exit Conditions

For all gradient-based line-search methods, a step is only meaningful if the proposed direction is an ascent direction:

$$\nabla \text{ELBO}(\xi^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (35)$$

Exit conditions are applied uniformly across methods using gradient norm, relative ELBO change, parameter change, and maximum iteration count. The detailed implementation of these conditions is given in [Section 6](#).

6 Test Cases and Implementation Details

This section defines the two numerical test cases and gives the implementation details used to apply the optimisation methods. The code snippets shown here define the data-generation components of the experiments. The full implementation structure, including ELBO evaluation, CAVI, gradient ascent, Newton’s method, BFGS, and Gibbs sampling, is provided in [Appendix C](#).

6.1 Test Case 1: Linear Bayesian Regression

The first test case is a linear Bayesian regression problem. It provides the baseline setting where the data-generating process matches the assumed regression model:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad (36)$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_e^2). \quad (37)$$

The design matrix is

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}. \quad (38)$$

This case is used to check whether each optimisation method can reproduce the CAVI solution and to measure posterior variance collapse in the simplest regression setting.

```
def generate_linear_case(n=100, sigma=0.5, seed=1):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x])
    beta_true = np.array([1.0, 2.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true
```

6.2 Test Case 2: Quadratic Bayesian Regression

The second test case is a quadratic regression problem. The response is generated by

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i, \quad (39)$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_e^2). \quad (40)$$

Although the relationship between x_i and y_i is nonlinear, the model remains linear in the coefficient vector $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2)^T$. The nonlinear feature is handled through the expanded design matrix

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}. \quad (41)$$

This test case is more demanding because the additional feature changes the geometry of the design matrix and can increase coupling among the variational parameters.

```
def generate_quadratic_case(n=100, sigma=0.5, seed=2):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x, x**2])
    beta_true = np.array([1.0, 2.0, -1.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true
```

6.3 Code Structure

For each test case, the same workflow is applied:

1. Generate $\mathbf{X}$, $\mathbf{y}$, and β_{true} .
2. Fit the CAVI baseline and record the ELBO trajectory.
3. Fit the gradient ascent, Newton, and BFGS implementations using the same ELBO objective.
4. Run the Gibbs sampler reference implementation.
5. Compare posterior means, posterior standard deviations, convergence speed, and runtime.

```
for case_name, generator in {
    "linear": generate_linear_case,
    "quadratic": generate_quadratic_case,
}.items():
    X, y, beta_true = generator()
    results[case_name] = run_all_methods(X, y, beta_true)
```

6.4 Implementation Specifications

This section provides complete algebraic detail for all numerical transformations required for implementation. These specifications ensure the final report can reference exact formulas without derivation.

1. Cholesky Parameterisation

Forward transformation (constrained to unconstrained):

Given $\Sigma_\beta \succ 0$, compute Cholesky decomposition:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (42)$$

where $\mathbf{L}$ is lower triangular with $L_{ii} > 0$.

Extract unconstrained parameters:

$$\ell_{ii} = \log L_{ii} \quad \text{for } i = 1, \dots, p \quad (43)$$

$$\ell_{ij} = L_{ij} \quad \text{for } i > j \quad (44)$$

Inverse transformation (unconstrained to constrained):

Given unconstrained vector ℓ , reconstruct $\mathbf{L}$:

$$L_{ii} = e^{\ell_{ii}} \quad \text{for } i = 1, \dots, p \quad (45)$$

$$L_{ij} = \ell_{ij} \quad \text{for } i > j \quad (46)$$

$$L_{ij} = 0 \quad \text{for } i < j \quad (47)$$

Then compute:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (48)$$

Gradient transformation (chain rule):

Let $\mathcal{L}(\xi)$ be the ELBO expressed in unconstrained parameters. By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} \frac{\partial \mathcal{L}}{\partial \Sigma_{\beta,km}} \frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ij}} \quad (49)$$

For diagonal entries ($i = j$):

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ii}} = \frac{\partial}{\partial \ell_{ii}} \left[\sum_s L_{ks} L_{ms} \right] \quad (50)$$

$$= \frac{\partial L_{ki}}{\partial \ell_{ii}} L_{mi} + L_{ki} \frac{\partial L_{mi}}{\partial \ell_{ii}} \quad (51)$$

Since $L_{ii} = e^{\ell_{ii}}$ and $L_{ij} = \ell_{ij}$ for $i \neq j$:

$$\frac{\partial L_{ki}}{\partial \ell_{ii}} = \begin{cases} e^{\ell_{ii}} = L_{ii} & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases} \quad (52)$$

Therefore:

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ii}} = \delta_{ki} L_{ii} L_{mi} + \delta_{mi} L_{ki} L_{ii} \quad (53)$$

$$= L_{ii} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (54)$$

For off-diagonal Cholesky entries ($i > j$):

$$\frac{\partial L_{ki}}{\partial \ell_{ij}} = \delta_{ki} \delta_{ji} = \begin{cases} 1 & \text{if } k = i, s = j \\ 0 & \text{otherwise} \end{cases} \quad (55)$$

Thus:

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ij}} = \delta_{ki} L_{mj} + \delta_{mi} L_{kj} \quad (56)$$

Complete gradient formula:

Let $\mathbf{G} = \nabla_{\Sigma_{\beta}} \mathcal{L}$ be the gradient with respect to the covariance matrix. Then:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ii}} = L_{ii} \sum_{k,m} G_{km} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (57)$$

$$= L_{ii} \left(2 \sum_m G_{im} L_{mi} - G_{ii} L_{ii} \right) \quad (58)$$

$$= 2 L_{ii} (\mathbf{G}\mathbf{L})_{ii} - L_{ii}^2 G_{ii} \quad (59)$$

$$= L_{ii} (2(\mathbf{G}\mathbf{L})_{ii} - L_{ii} G_{ii}) \quad (60)$$

For off-diagonal:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} G_{km} (\delta_{ki} L_{mj} + \delta_{mi} L_{kj}) \quad (61)$$

$$= \sum_m G_{im} L_{mj} + \sum_k G_{km} L_{kj} \quad (62)$$

$$= (\mathbf{G}\mathbf{L})_{ij} + (\mathbf{G}^T \mathbf{L})_{ij} \quad (63)$$

$$= ((\mathbf{G} + \mathbf{G}^T) \mathbf{L})_{ij} \quad (64)$$

Implementation: Compute $\mathbf{G} = \nabla_{\Sigma_{\beta}} \text{ELBO}$, then transform to Cholesky space using the above formulas.

2. Log-Space Parameterisation

Forward transformation:

$$\eta_a = \log a_e \quad (65)$$

$$\eta_b = \log b_e \quad (66)$$

Inverse transformation:

$$a_e = e^{\eta_a} \quad (67)$$

$$b_e = e^{\eta_b} \quad (68)$$

Gradient transformation with Jacobian adjustment:

By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \frac{\partial a_e}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \cdot e^{\eta_a} = a_e \cdot \frac{\partial \mathcal{L}}{\partial a_e} \quad (69)$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial \eta_b} = b_e \cdot \frac{\partial \mathcal{L}}{\partial b_e} \quad (70)$$

Implementation: Compute gradients $\nabla_{a_e} \text{ELBO}$ and $\nabla_{b_e} \text{ELBO}$, then multiply by a_e and b_e respectively.

3. Initialisation Strategy

Primary initialisation (from OLS):

$$\boldsymbol{\mu}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{OLS estimate}) \quad (71)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{I}_p \quad (\text{moderate initial uncertainty}) \quad (72)$$

$$a_e^{(0)} = \alpha_e + \frac{n}{2} \approx 51 \quad (\text{data-informed shape}) \quad (73)$$

$$b_e^{(0)} = 1.0 \quad (\text{vague initial rate}) \quad (74)$$

Rationale: OLS provides good starting point for $\boldsymbol{\mu}_\beta$; identity covariance avoids overconfidence; a_e reflects data contribution.

Alternative initialisation (zero-centred):

$$\boldsymbol{\mu}_\beta^{(0)} = \mathbf{0}_p \quad (75)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \quad (\text{frequentist uncertainty}) \quad (76)$$

$$a_e^{(0)} = \alpha_e + 1 = 2 \quad (77)$$

$$b_e^{(0)} = \gamma_e + 1 = 2 \quad (78)$$

Cholesky initialisation:

Given initial $\boldsymbol{\Sigma}_\beta^{(0)}$, compute Cholesky:

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{L}^{(0)} (\mathbf{L}^{(0)})^T \quad (79)$$

$$\ell_{ii}^{(0)} = \log L_{ii}^{(0)} \quad (80)$$

$$\ell_{ij}^{(0)} = L_{ij}^{(0)} \quad \text{for } i > j \quad (81)$$

Log-space initialisation:

$$\eta_a^{(0)} = \log a_e^{(0)} \quad (82)$$

$$\eta_b^{(0)} = \log b_e^{(0)} \quad (83)$$

4. Line Search: Armijo Backtracking

Algorithm 1 Armijo Backtracking Line Search

Require: Current point $\xi^{(t)}$, search direction $\mathbf{d}^{(t)}$, gradient $\nabla \mathcal{L}^{(t)}$

Require: Parameters: $c_1 = 10^{-4}$ (sufficient decrease), $\rho = 0.5$ (backtracking factor), $\alpha_{\max} = 1.0$

Ensure: Step size α_t satisfying Armijo condition

```

1:  $\alpha \leftarrow \alpha_{\max}$ 
2:  $\text{max\_iter} \leftarrow 50$ 
3:  $k \leftarrow 0$ 
4: while  $k < \text{max\_iter}$  do
5:   Compute trial point:  $\xi_{\text{trial}} \leftarrow \xi^{(t)} + \alpha \mathbf{d}^{(t)}$ 
6:   Evaluate:  $\mathcal{L}_{\text{trial}} \leftarrow \mathcal{L}(\xi_{\text{trial}})$ 
7:   Compute sufficient decrease threshold:  $\mathcal{L}_{\text{threshold}} \leftarrow \mathcal{L}(\xi^{(t)}) + c_1 \alpha (\nabla \mathcal{L}^{(t)})^T \mathbf{d}^{(t)}$ 
8:   if  $\mathcal{L}_{\text{trial}} \geq \mathcal{L}_{\text{threshold}}$  then
9:     return  $\alpha$  // Armijo condition satisfied
10:  end if
11:   $\alpha \leftarrow \rho \alpha$  // Reduce step size
12:   $k \leftarrow k + 1$ 
13: end while
14: Warning: Line search failed, return  $\alpha = 10^{-8}$  (damped step)
15: return  $\alpha$ 

```

Key parameters:

- $c_1 = 10^{-4}$: Standard value for sufficient decrease (Nocedal & Wright)
- $\rho = 0.5$: Halves step size at each backtrack iteration
- $\alpha_{\max} = 1.0$: Start with full Newton step (for Newton and BFGS)
- For gradient ascent: may use $\alpha_{\max} = 0.1$ initially

5. Newton's Method Regularisation

When Hessian $\mathbf{H} = \nabla^2 \text{ELBO}$ is not negative definite (for maximisation), regularise:

$$\mathbf{H}_{\text{reg}} = \mathbf{H} - \lambda \mathbf{I}_d \quad (84)$$

Regularisation parameter selection:

1. Compute eigenvalue¹¹ decomposition: $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$ (expensive, $O(d^3)$)
2. Or use Cholesky attempt: try $\mathbf{H} = \mathbf{R}^T \mathbf{R}$; if fails, Hessian not positive definite
3. Simpler approach (Levenberg damping):

$$\lambda = \begin{cases} 0 & \text{if Cholesky succeeds (Hessian negative definite)} \\ \max(10^{-6}, -2\lambda_{\min}) & \text{otherwise} \end{cases} \quad (85)$$

where $\lambda_{\min}$ is smallest eigenvalue of $\mathbf{H}$.

¹¹**Eigenvalues** are numbers $\lambda_1, \lambda_2, \dots$ that describe how a matrix scales space along its natural axes (eigenvectors). For a symmetric matrix like Σ_β : all eigenvalues > 0 means positive definite; all eigenvalues ≥ 0 means positive semidefinite; any eigenvalue < 0 means not a valid covariance matrix. They are the stretching factors of the matrix in each principal direction.

Practical implementation (without eigenvalue computation):

Start with $\lambda = 0$. If Cholesky of $-\mathbf{H}$ fails:

$$\lambda \leftarrow 10^{-4} \quad (86)$$

$$\mathbf{H}_{\text{reg}} \leftarrow \mathbf{H} - \lambda \mathbf{I}_d \quad (87)$$

Repeat with $\lambda \leftarrow 10\lambda$ until Cholesky succeeds or $\lambda > 10^3$ (failure).

Newton search direction:

$$\mathbf{d}^{(t)} = -\mathbf{H}_{\text{reg}}^{-1} \nabla \mathcal{L}^{(t)} \quad (88)$$

For maximisation, this should be an ascent direction: $(\nabla \mathcal{L})^T \mathbf{d} > 0$.

6. Convergence Criteria

Implement multiple stopping conditions (logical OR):

1. **Gradient norm:** $\|\nabla \mathcal{L}^{(t)}\|_2 < \epsilon_g = 10^{-6}$

Indicates first-order optimality (stationary point reached).

2. **Relative ELBO change:**

$$\frac{|\mathcal{L}^{(t)} - \mathcal{L}^{(t-1)}|}{1 + |\mathcal{L}^{(t-1)}|} < \epsilon_{\text{rel}} = 10^{-8} \quad (89)$$

Prevents premature termination when ELBO is large in magnitude.

3. **Parameter change:**

$$\|\boldsymbol{\xi}^{(t)} - \boldsymbol{\xi}^{(t-1)}\|_2 < \epsilon_x = 10^{-6} \quad (90)$$

Catches slow convergence in parameter space.

4. **Maximum iterations:** $t \geq t_{\text{max}} = 1000$

Safety check to prevent infinite loops.

Convergence flag assignment:

- Status 0: Gradient norm criterion (ideal)
- Status 1: Relative ELBO change (acceptable)
- Status 2: Parameter change (acceptable but slower)
- Status 3: Maximum iterations (failure to converge)

7. Numerical Gradient Verification

For debugging, verify analytical gradients using finite differences:

Central difference formula:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} \approx \frac{\mathcal{L}(\boldsymbol{\xi} + h \mathbf{e}_i) - \mathcal{L}(\boldsymbol{\xi} - h \mathbf{e}_i)}{2h} \quad (91)$$

where $\mathbf{e}_i$ is the i -th standard basis vector and h is step size.

Step size selection:

For float64 precision ($\epsilon_{\text{machine}} \approx 2.2 \times 10^{-16}$):

$$h = \sqrt{\epsilon_{\text{machine}}} \max(1, |\xi_i|) \approx 1.5 \times 10^{-8} \max(1, |\xi_i|) \quad (92)$$

Error metric:

$$\text{error}_i = \frac{|g_i^{\text{analytical}} - g_i^{\text{numerical}}|}{\max(|g_i^{\text{analytical}}|, |g_i^{\text{numerical}}|, 10^{-8})} \quad (93)$$

Acceptable if $\text{error}_i < 10^{-5}$ for all i .

8. Failure Mode Handling

Common failure modes and recovery:

1. Matrix not positive definite:

- Detection: Cholesky decomposition fails
- Recovery: Add regularisation $\Sigma_\beta \leftarrow \Sigma_\beta + 10^{-6} \mathbf{I}_p$

2. Line search exhaustion:

- Detection: 50 backtracking iterations without Armijo satisfaction
- Recovery: Accept tiny step $\alpha = 10^{-8}$ and continue; if recurring, terminate

3. ELBO decrease:

- Detection: $\mathcal{L}^{(t)} < \mathcal{L}^{(t-1)}$ after line search
- Recovery: Reduce step size aggressively or revert to previous parameters

4. Parameter divergence:

- Detection: $\|\xi^{(t)}\|_2 > 10^{10}$ or any $|\xi_i| > 10^6$
- Recovery: Reinitialise from OLS estimates

5. NaN/Inf in ELBO or gradient:

- Detection: `np.isnan()` or `np.isinf()` checks
- Recovery: Reduce step size or add numerical safeguards (min/max clipping)

7 Evaluation Criteria

1. Convergence: All methods reach ELBO optimum within tolerance

2. Correctness:

- CAVI correctness: Gradient/Newton/BFGS implementation is correct if it reaches the same ELBO optimum / variational fixed point as CAVI (same VI objective, same variational family)
- MCMC correctness: VI is assessed against MCMC as a posterior-accuracy benchmark (means, intervals, calibration), not expected to match exactly because VI is approximate

3. Performance hierarchy:

- CAVI likely fastest for this conjugate problem

- BFGS expected to be competitive general-purpose method
- Newton may struggle with indefinite Hessian
- MCMC/Gibbs expected slowest computationally, but best for posterior uncertainty accuracy (gold-standard benchmark, not a direct ELBO-optimiser competitor)

4. Numerical verification:

- Analytical gradients match finite differences within 10^{-5} relative error
- All methods reach same ELBO value (within 10^{-4})
- Posterior means agree with CAVI and MCMC within 2 standard errors

8 Results and Discussion

The results are organised around the primary question of posterior variance collapse. For each test case, the optimisation methods should be compared using both optimisation diagnostics and posterior-uncertainty diagnostics.

The optimisation diagnostics include final ELBO value, number of iterations, total runtime, line-search behaviour, and convergence status. These quantities determine whether each method successfully solves the numerical optimisation problem.

The posterior diagnostics include posterior mean accuracy and posterior standard deviation relative to the Gibbs sampler. The key variance-collapse diagnostic is the ratio

$$\frac{\sigma_{\text{VB}}}{\sigma_{\text{Gibbs}}}. \quad (94)$$

A ratio below one indicates that the variational posterior is narrower than the Gibbs reference. The comparison should therefore distinguish between numerical convergence and uncertainty accuracy: a method may converge successfully while still producing an under-dispersed variational posterior.

The linear test case provides the baseline comparison. The quadratic test case examines whether the same optimisation and variance-collapse behaviour persists when the design matrix is expanded to include a nonlinear feature.

9 Conclusion

This report investigates posterior variance collapse in mean-field variational Bayesian regression using the gradient-based optimisation methods taught in ENEL445. The problem is formulated as ELBO maximisation, with CAVI used as the analytical baseline and gradient ascent, Newton’s method, and BFGS used as general-purpose optimisation methods.

The central issue is not only whether the methods converge, but whether the resulting variational approximation provides reliable posterior uncertainty. Cholesky and log-space reparameterisations are used to ensure that covariance and Gamma parameters remain valid throughout optimisation. Linear and quadratic Bayesian regression test cases are used to compare convergence behaviour, computational cost, posterior mean accuracy, and posterior variance behaviour.

The report therefore uses optimisation methods as both computational tools and diagnostic tools. Even when ELBO optimisation is successful, the mean-field approximation may still underestimate posterior variance. This distinction is essential: convergence of the ELBO does not by itself guarantee accurate Bayesian uncertainty quantification.

ewpage

References

- Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877. <https://doi.org/10.1080/01621459.2017.1285773>
- Wainwright, M. J., & Jordan, M. I. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2), 1–305. <https://www.cs.columbia.edu/~blei/fogm/2020F/readings/WainwrightJordan2008.pdf>

References

- Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877. <https://doi.org/10.1080/01621459.2017.1285773>
- Wainwright, M. J., & Jordan, M. I. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2), 1–305. <https://www.cs.columbia.edu/~blei/fogm/2020F/readings/WainwrightJordan2008.pdf>

Appendix A VB Posterior Variance Collapse

Why Variance Collapse Happens

Variational inference finds $q(\boldsymbol{\theta})$ by minimising the KL divergence from q to the true posterior $p(\boldsymbol{\theta} \mid \mathbf{y})$:

$$\text{KL}[q\|p] = \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid \mathbf{y})} d\boldsymbol{\theta} \quad (95)$$

This is called the *forward* (or *exclusive*) KL divergence. The critical asymmetry: this expression is large when q places mass where p is small, but contributes *zero* when q places no mass where p is large. The optimiser therefore drives q to avoid regions where p is small — but has no incentive whatsoever to cover regions where p is large. The result is a q that locks onto the high-probability core of p and ignores the tails, systematically underestimating posterior variance.

This is sometimes called *zero-forcing* behaviour: q is forced to be zero in low-probability regions of p , but is not forced to cover high-probability regions. The reverse direction $\text{KL}[p\|q]$ — called the *reverse* or *inclusive* KL — would instead force q to cover all regions where p has mass, producing over-dispersed approximations. Neither direction recovers the true posterior exactly; standard VI picks the under-dispersed failure mode.

Mean-field factorisation compounds the problem: by forcing $q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta})q(\tau_e)$, any true posterior correlation between $\boldsymbol{\beta}$ and τ_e is set to zero by assumption. Posterior correlations that would otherwise widen the marginals are simply removed, squeezing the approximation tighter.

Observation

Comparison of the VB posterior against the MCMC (Gibbs sampler) gold standard confirms this: the mean-field VB approximation systematically underestimates posterior variance — a well-known failure mode of mean-field variational inference. This will be demonstrated empirically as the first Python task, by running both methods on the same synthetic dataset and computing the SD ratio (VB σ / Gibbs σ) for each parameter. Ratios below 1.0 confirm collapse; the magnitude of the shortfall motivates the constraint-handling approaches below.

Three Approaches to Prevent Collapse

1. Informative Prior on τ_e

Set non-zero prior hyperparameters (α_e, γ_e) to place a lower bound on the posterior variance. A tight prior on τ_e prevents the precision from being driven too high, which in turn keeps $\boldsymbol{\Sigma}_\beta$ from collapsing.

Mechanism: The CAVI update for b_e is:

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \boldsymbol{\Sigma}_\beta)] \quad (96)$$

With $\gamma_e > 0$, b_e has a minimum floor, bounding $\mathbb{E}[\tau_e] = a_e/b_e$ from above.

Limitation: Changes the model; requires prior elicitation.

2. Variance Lower Bound Constraint

After each CAVI update, clip Σ_β so its diagonal never falls below a minimum threshold $\sigma_{\min}^2$:

$$\Sigma_{\beta,jj} \leftarrow \max(\Sigma_{\beta,jj}, \sigma_{\min}^2) \quad \forall j \quad (97)$$

This was already noted in the Constraints section as an optional minimum variance constraint. In code:

```
Sigma_diag = np.diag(self.Sigma_beta)
np.fill_diagonal(self.Sigma_beta,
    np.maximum(Sigma_diag, sigma_min))
```

Limitation: Ad hoc; does not arise from a principled objective modification.

3. Entropy Regularisation of the ELBO

Add a regularisation term proportional to the entropy $H[q]$ of the variational distribution to the objective:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda \cdot H[q(\beta)] \quad (98)$$

For $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, the Gaussian entropy is:

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \Sigma_\beta) = \frac{p}{2} (1 + \log 2\pi) + \frac{1}{2} \log \det(\Sigma_\beta) \quad (99)$$

The regularised gradient with respect to Σ_β gains an additional term:

$$\nabla_{\Sigma_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\Sigma_\beta} \mathcal{L} + \frac{\lambda}{2} \Sigma_\beta^{-1} \quad (100)$$

This directly rewards larger Σ_β , counteracting the collapse tendency. The hyperparameter $\lambda > 0$ controls the strength of the correction.

This is the most principled of the three approaches and is directly amenable to the gradient-based optimisation methods (gradient ascent, Newton, BFGS) already implemented for this paper.

Selected Approach for This Paper

Entropy regularisation (Approach 3) will be implemented and compared against the unregularised ELBO. The regularisation parameter λ will be treated as a hyperparameter and tuned by comparing posterior interval coverage against the MCMC reference. Results will be presented as a secondary comparison alongside the main method comparison.

Appendix B Cholesky Reparameterisation: Structural Constraint Satisfaction

The Problem with Direct Optimisation

The covariance matrix Σ_β must satisfy $\Sigma_\beta \succ 0$ (symmetric positive definite) at every iteration. If the optimiser updates Σ_β directly, a gradient step of the form

$$\Sigma_\beta^{(t+1)} = \Sigma_\beta^{(t)} + \alpha_t \nabla_{\Sigma_\beta} \mathcal{L} \quad (101)$$

can produce a matrix that is *not* positive definite. Standard gradient-based methods operate on unconstrained Euclidean space and have no built-in mechanism to respect the positive-definiteness constraint — the constraint is simply violated as a by-product of the update, leading to numerical failure (negative variances, failed Cholesky in later steps, NaN in the ELBO).

The Cholesky Solution: Structural Guarantee

Cholesky decomposition eliminates the constraint by parameterising Σ_β in terms of a lower-triangular factor $\mathbf{L}$:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T, \quad L_{ii} > 0 \quad \forall i \quad (102)$$

The fundamental insight is that **every** lower-triangular matrix $\mathbf{L}$ with strictly positive diagonal entries produces a valid symmetric positive definite matrix:

- **Symmetry:** $(\mathbf{L}\mathbf{L}^T)^T = (\mathbf{L}^T)^T \mathbf{L}^T = \mathbf{L}\mathbf{L}^T$ — automatically symmetric.
- **Positive definiteness:** For any non-zero $\mathbf{v} \in \mathbb{R}^p$,

$$\mathbf{v}^T \mathbf{L}\mathbf{L}^T \mathbf{v} = \|\mathbf{L}^T \mathbf{v}\|^2 > 0 \quad (103)$$

provided $\mathbf{L}^T$ has full column rank, which holds whenever all $L_{ii} > 0$ (since $\mathbf{L}$ is lower triangular, its rank equals the number of non-zero diagonal entries).

It is therefore *structurally impossible* to produce an invalid Σ_β by updating $\mathbf{L}$, regardless of how large or in what direction the gradient step is taken. The constraint is not enforced by a penalty or projection — it is encoded into the *form* of the parameterisation itself.

Unconstrained Optimisation Over ℓ

The remaining difficulty is that $L_{ii} > 0$ is still an inequality constraint. This is resolved by log-parameterising the diagonal:

$$L_{ii} = e^{\ell_{ii}}, \quad \ell_{ii} \in \mathbb{R} \quad (\text{diagonal}) \quad (104)$$

$$L_{ij} = \ell_{ij}, \quad \ell_{ij} \in \mathbb{R} \quad (\text{lower off-diagonal}) \quad (105)$$

Since $e^{\ell_{ii}} > 0$ for all $\ell_{ii} \in \mathbb{R}$, the optimisation is now over a *fully unconstrained* parameter vector $\ell \in \mathbb{R}^{p(p+1)/2}$. No projection, no clipping, no penalty — any point in $\mathbb{R}^{p(p+1)/2}$ maps to a valid $\mathbf{L}$, and hence a valid $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$.

Analogy with Log-Space Parameterisation

This is the same idea used for the Gamma parameters $a_e, b_e > 0$: instead of constraining $a_e > 0$ directly, we optimise over unconstrained $\eta_a = \log a_e \in \mathbb{R}$ and recover $a_e = e^{\eta_a}$, which is always positive. Cholesky + log-diagonal does the same thing for the cone of symmetric positive definite matrices: it provides a smooth bijection between unconstrained Euclidean space and the constraint set, so standard optimisation machinery applies without modification.

Cholesky reparameterisation is a numerical implementation technique. Cholesky is the domain-specific choice for satisfying the constraint $\Sigma_\beta \succ 0$ when applying those methods to this problem. The set of symmetric positive definite matrices is not a flat subspace of $\mathbb{R}^{p \times p}$: it has a curved boundary, and a straight-line gradient step can cross outside it, producing an invalid covariance matrix. Cholesky reparameterisation handles this by converting the constraint into a smooth bijection between the valid set and unconstrained $\mathbb{R}^{p(p+1)/2}$, so standard gradient steps always remain valid. It is standard practice in variational inference and Bayesian computation.

Appendix C Python Implementation Structure

The implementation follows a top-down sequential process across five stages. Each stage depends on the outputs of the previous one and saves its results to parquet before proceeding, so that any stage can be re-run independently without repeating expensive computation.

```
# Stage 1: Data generation
#   - generate_linear_case -> data/linear_data.parquet
#                               figs/linear_data_scatter.png

# Stage 2: ELBO evaluation
#   - unpack unconstrained parameters (mu_beta, L_chol, log_ae, log_be)
#   - reconstruct Sigma_beta via Cholesky: Sigma = L @ L.T
#   - reconstruct a_e, b_e via exp transforms
#   - evaluate standard or entropy-regularised ELBO
#   - verify analytical gradient against finite differences (<= 1e-5)
#                               figs/linear_elbo_gradient_check.png
#                               figs/linear_elbo_landscape.png
#                               results/tables/linear_gradient_check.tex

# Stage 3: Optimisation methods
#   - run_cavi
#   - run_gradient_ascent      (step-size history recorded)
#   - run_newton                (Hessian condition number tracked)
#   - run_bfgs                 (via scipy.optimize.minimize)
#                               -> data/linear_opt.parquet
#                               figs/linear_elbo_convergence.png
#                               figs/linear_gradient_stepsize.png
#                               figs/linear_newton_condition.png

# Stage 4: Reference sampler
#   - run_gibbs_sampler        (3 chains, configurable burn-in)
#                               -> data/linear_gibbs.parquet
#                               figs/linear_gibbs_trace_{beta0,beta1,taue}.png
#                               figs/linear_gibbs_acf_{beta0,beta1,taue}.png
#                               figs/linear_gibbs_rhat.png

# Stage 5: Diagnostics and comparison
#   - final ELBO per method
#   - number of iterations / runtime / convergence status
#   - posterior mean error: |mu_VB - mu_Gibbs|
#   - posterior SD ratio: sigma_VB / sigma_Gibbs (variance collapse)
#   - VB vs Gibbs overlay: figs/linear_comparison_{beta0,beta1,taue}.png
#   - VB vs Gibbs density: figs/linear_vb_gibbs_{beta0,beta1,taue}.png
#                               -> figs/linear_sd_ratios.png
#                               figs/linear_mean_errors.png
#                               results/tables/linear_performance.tex
#                               results/tables/linear_posterior_summary.tex
```

The three Python files that implement the above are described below.

File: vb_algorithms_py.py

- **ELBO evaluation** — unpacks unconstrained parameter vector, reconstructs Σ_β via Cholesky lower-triangular factor, reconstructs a_e and b_e via exp transforms, evaluates standard or entropy-regularised ELBO.
- **Analytical gradient** — computes $\nabla_\theta \mathcal{L}$ with respect to all unconstrained parameters; verified against finite differences to relative error $\leq 10^{-5}$.
- **Hessian** — computes the Hessian of the ELBO for Newton's method; condition number tracked per iteration.
- **CAVI baseline** — coordinate ascent updates for $q(\beta)$ and $q(\tau_e)$.
- **Gradient ascent** — fixed or line-search step size; step-size history recorded.
- **Newton's method** — Hessian-based updates; falls back on gradient step if Hessian indefinite.
- **BFGS** — quasi-Newton method via `scipy.optimize.minimize`.
- **Gibbs sampler** — blocked sampler for $\beta \mid \tau_e, \mathbf{y}$ and $\tau_e \mid \beta, \mathbf{y}$; multiple chains with configurable burn-in.

File: vb_utils_py.py

- **Data generation** — `generate_linear_case`: generates design matrix $\mathbf{X}$, response $\mathbf{y}$, and true parameter values for the linear Bayesian regression test case.
- **Parquet I/O** — `save_results_parquet`: writes numerical results to `results/data/*.parquet` as the source of truth; `load_results_parquet`: reloads without rerunning computation.
- **Table rendering** — `render_table_tex`: reads a polars DataFrame, formats values, writes a `tabularray (tblr)` snippet to `results/tables/*.tex` for `\input{}` inclusion in the report; also compiles a standalone preview PNG for inline notebook display.
- **Plot helpers** — `save_fig`: saves to `figs/` with consistent style; helpers for scatter plots, KDE overlays, bar charts, ACF, trace plots, ELBO curves, step-size histories, Hessian condition-number histories.

Notebook: linear_run.ipynb

The notebook executes the linear case in five stages, one `# %%` cell block per stage. Each stage saves its outputs before proceeding so that individual stages can be re-run independently.

1. **Data generation** — generates and saves `linear_data.parquet`; produces `linear_data_scatter.png` (1 figure).
2. **ELBO evaluation** — verifies analytical gradient against finite differences at the CAVI optimum; produces gradient-check bar and ELBO landscape slice (2 figures). Must pass before any optimiser is run.
3. **Optimisation methods** — runs CAVI, gradient ascent, Newton, and BFGS; saves `linear_opt.parquet`; produces ELBO convergence curves, step-size history, and Hessian condition-number history (3 figures).
4. **Reference sampler** — runs Gibbs sampler (3 chains); saves `linear_gibbs.parquet`; produces trace plots, ACF plots, $\hat{R}$ bar, and per-component distribution plots (7 figures).

5. **Diagnostics and comparison** — loads all parquet files; produces VB/Gibbs overlay plots, SD-ratio bar, and posterior mean-error bar; renders the three `tblr` tables to `results/tables/` and displays inline PNG previews (8 figures, 3 tables).

Total outputs for the linear case: **21 figures** saved to `figs/` and **3 tables** saved to `results/tables/`.

Appendix D Project Status and Course Alignment

The following material is retained from the working report draft as project-status and course-alignment documentation. It is placed in the appendix so that the main body remains report-focused while preserving the planning details.

Progress To Date (as of 3 May 2026)

Completed Work

1. **Problem formulation:** Optimisation problem fully specified with decision variables, objective function (ELBO), constraints, and reparameterisation strategy (Cholesky + log-space).
2. **Mathematical derivations — COMPLETE:**
 - ✓ Complete ELBO derivation from first principles (13 pages, documented)
 - ✓ CAVI algorithm with closed-form updates and convergence proof
 - ✓ Analytical gradient expressions for all 11 variational parameters
 - ✓ Gradient transformations for Cholesky and log-space parameterisations with complete chain-rule algebra
 - ✓ Hessian block structure for Newton’s method
 - ✓ BFGS quasi-Newton formulation with update formulas
 - ✓ Penalty method formulation for constrained optimisation (alternative approach)
 - ✓ Initialisation strategy with OLS-based and zero-centred alternatives
 - ✓ Line search algorithm specification (Armijo backtracking with pseudocode)
 - ✓ Newton regularisation for indefinite Hessian
 - ✓ Convergence criteria with four stopping conditions
 - ✓ Numerical gradient verification procedure
 - ✓ Failure mode analysis with recovery strategies

All derivations documented in `vi_optimisation_solution.pdf` with complete algebraic steps suitable for reference in final report.

3. **Implementation readiness:** All mathematical specifications complete with step-by-step formulas. Ready to proceed with coding phase.

Preliminary Results

Mathematical analysis confirms:

- Problem is well-posed with unique global maximum (ELBO is concave in each coordinate block)
- CAVI updates guaranteed to converge monotonically

- Gradient-based methods require careful parameterisation to handle constraints
- Newton’s method may require regularisation away from optimum
- BFGS expected to be robust general-purpose method

Alignment with Course Requirements

The VI problem represents real computational challenges, making the comparison of optimisation methods valuable, relevant, and interesting. The project implements BFGS (one of the five required methods from Project 2 specification) along with two additional methods (Newton, gradient ascent) for comprehensive comparison.

Mathematical specifications are now complete and documented with full algebraic detail, enabling implementation to proceed systematically with clear validation checkpoints.

Timeline

Week	Dates	Milestone
7	24-31 Mar	CAVI baseline + gradient verification
8	1-7 Apr	Gradient ascent + Newton’s method
9	8-14 Apr	BFGS + convergence analysis
10	15-21 Apr	MCMC validation + sensitivity analysis
11	22-28 Apr	Final comparison + figure generation
12	May	Final report completion and submission